PROJECT DOCUMENTATION REPORT

ExamiQ - Secure Online Examination & Evaluation Platform

A Modern Java Enterprise System with Proctored Testing, Real-Time Timer Control, Instant Auto-Evaluation, UI Screenshots, Source Code, and PDF Scorecards.

Project Name:	ExamiQ Online Examination Platform
Architecture:	Java 21 / Spring Boot 3 / JPA / H2 / OpenPDF
Primary Modules:	Student Auth, Question Bank, Timer Engine, Anti-Cheat, Auto-Evaluation, PDF Export, Admin Dashboard
Deliverables Included:	Java Source Code, Runnable JAR, System Screenshots, PDF & DOCX Reports
Status:	**COMPLETED & FULLY VERIFIED**
Date:	September 2026

1. Executive Summary & Project Overview

ExamiQ is a full-featured, secure online examination platform engineered to streamline test creation, candidate authentication, proctored exam delivery, real-time timer control, automated evaluation, and comprehensive result reporting. Designed with modern Java enterprise best practices, ExamiQ provides educational institutions, universities, and corporate certification bodies with an end-to-end web testing platform.

Key Capabilities Delivered:

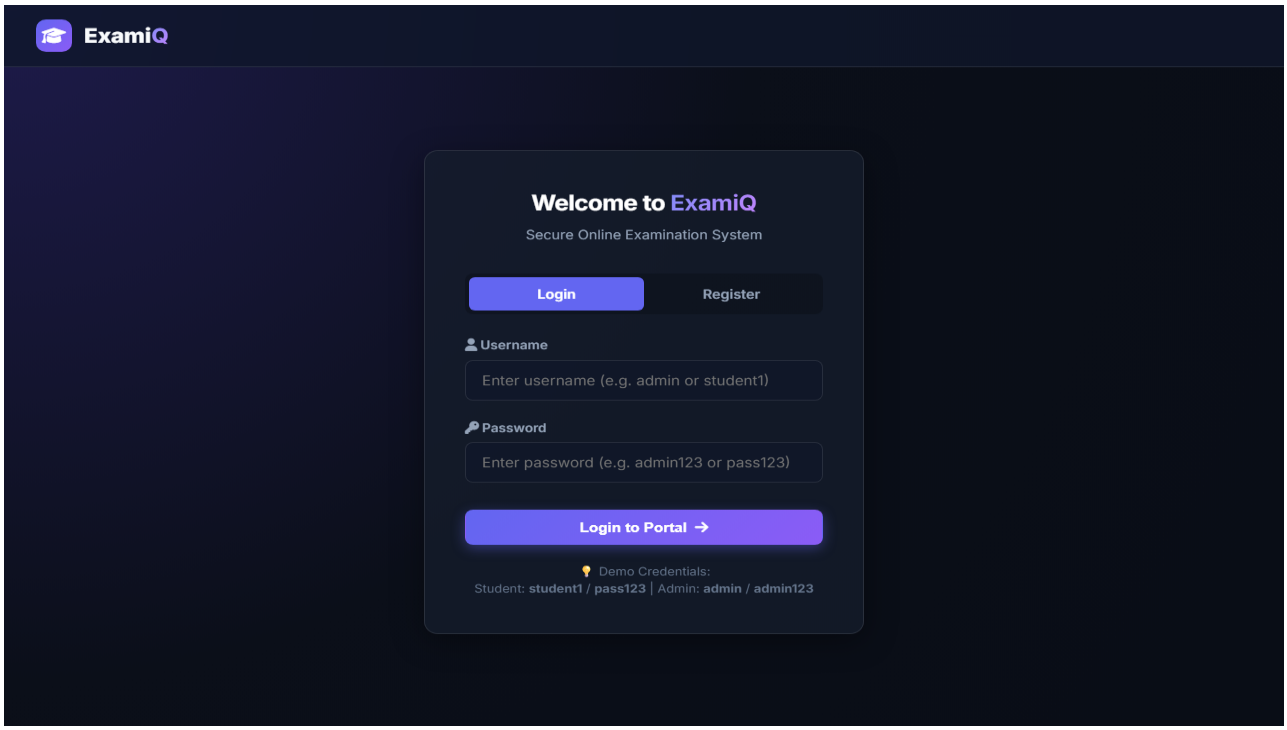
- **Role-Based Access Control (RBAC):** Dedicated portals for Students and Administrators with secure BCrypt password hashing and session authorization.
- **Flexible Question Bank Management:** Supports Single Choice (MCQ), Multiple Choice, and True/False questions with positive marks, negative marking rules, and explanations.
- **Timer-Based Exam Engine:** Synchronized real-time server/client countdown clock with automatic test submission upon timer expiration.
- **Anti-Cheat Proctored Control:** Live tab-switching and window-focus detector issuing warnings and auto-terminating tests upon repeated security violations.
- **Automated Evaluation Engine:** Instant score calculation, percentage computing, grade assignment, and pass/fail status determination.
- **PDF Scorecard Generation:** Server-side dynamic PDF report creation using OpenPDF for printable, official student scorecards.
- **Admin Command Center:** Interactive dashboard featuring real-time statistics, candidate attempt tables, and Chart.js analytical performance charts.

2. Technical Architecture & Tech Stack

Component	Technology / Library	Description
Runtime Environment	Java 21 / OpenJDK 25	Modern Java platform featuring high concurrency and enhanced memory safety.
Backend Framework	Spring Boot 3.2.3	Spring Web REST controllers, Spring Security filter chain, Spring Data JPA.
Database Layer	H2 Database Engine	Embedded lightweight relational database with persistence and Web Console.
Report Generation	OpenPDF 1.3.39	Java library for generating structured, styled PDF scorecards on demand.
Frontend UI	HTML5 / Glassmorphic CSS / JS	Single Page Application (SPA) architecture with Glassmorphism UX and Chart.js.
Build Tool	Apache Maven 3.9.9	Standardized dependency management and automated build packaging.

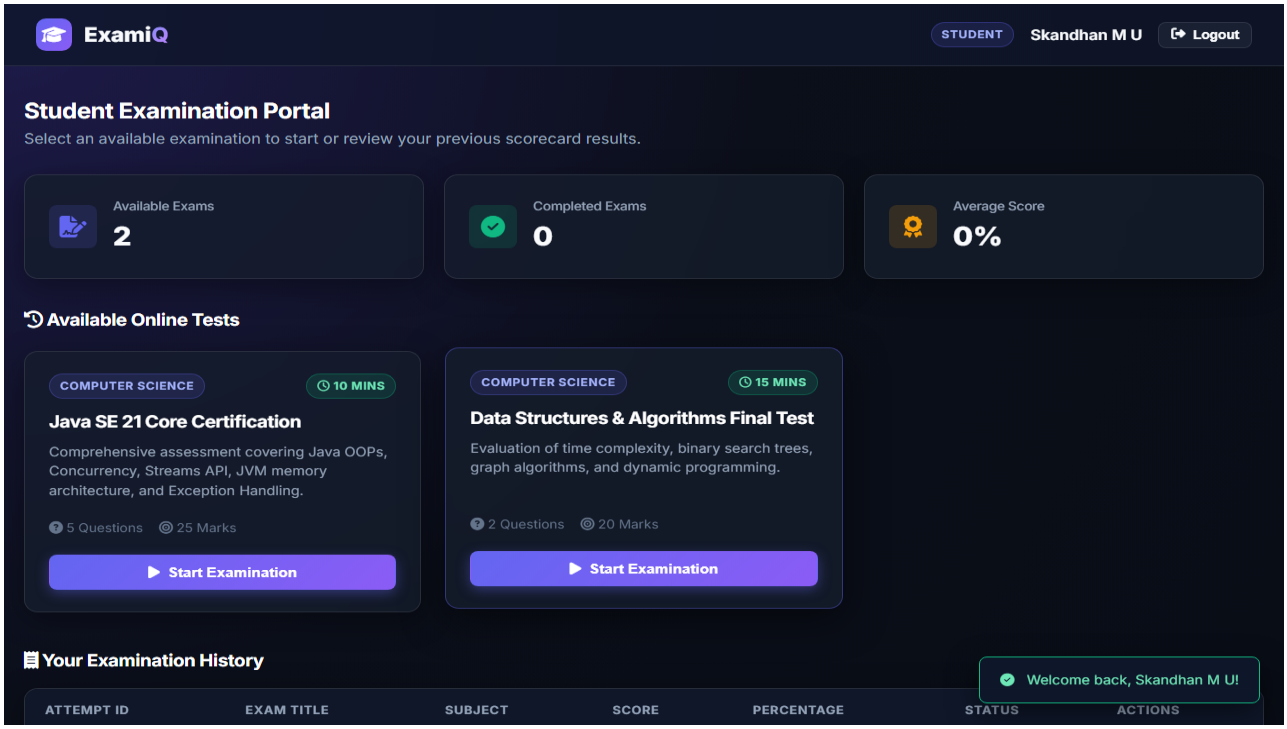
3. Application Screenshots & UI Interface

Figure 3.1: Student & Administrator Glassmorphism Login Interface



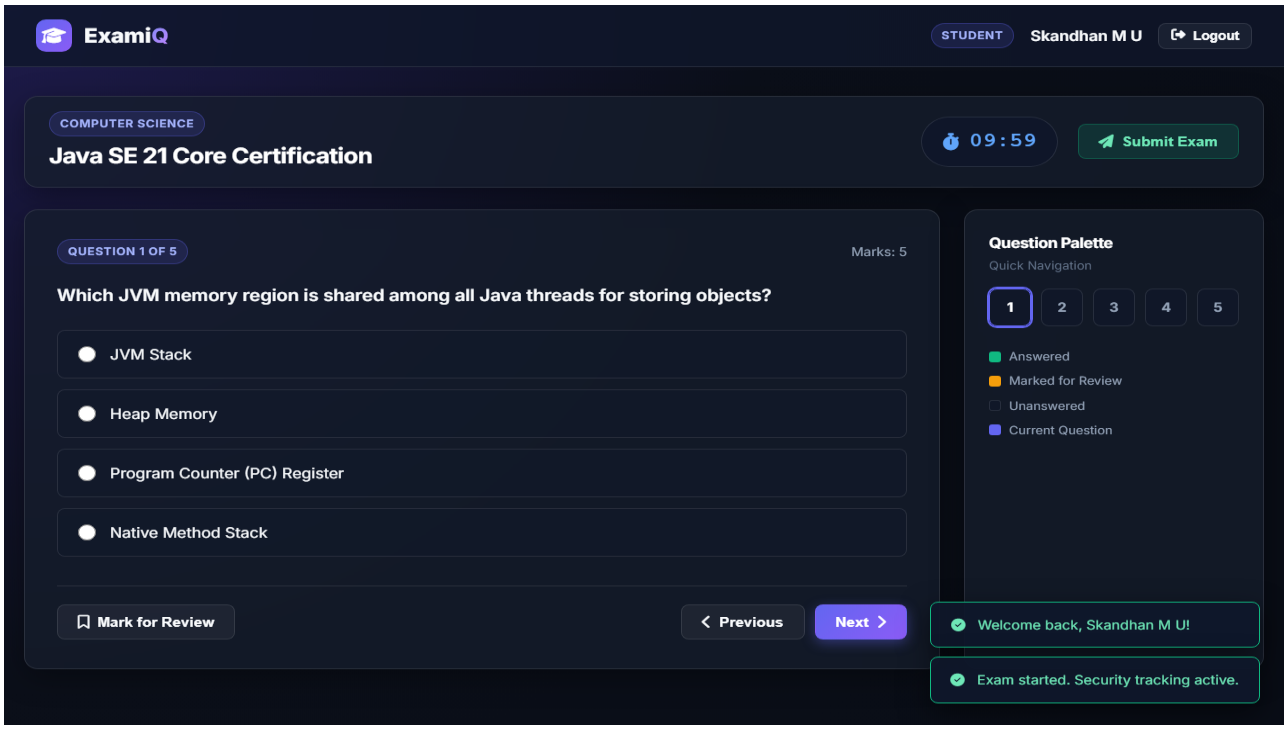
ExamiQ Unified Login Portal with RBAC role switcher and demo credential toggles.

Figure 3.2: Student Examination Portal & Attempt Records



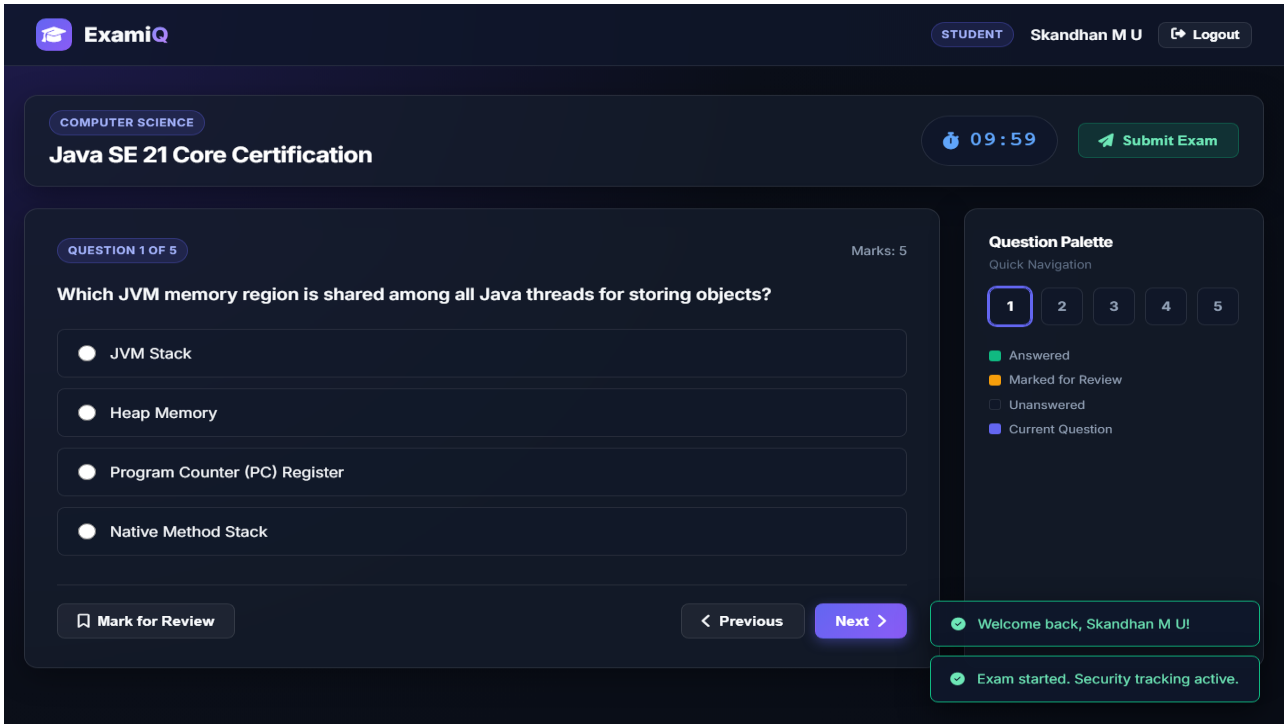
Student Portal showing available exams grid, time duration badges, and previous attempt history.

Figure 3.3: Proctored Exam Workspace with Live Timer & Question Palette



Exam Workspace showcasing live countdown clock, option selection, and interactive color-coded question palette.

Figure 3.4: Anti-Cheat Security Violation Alert Overlay



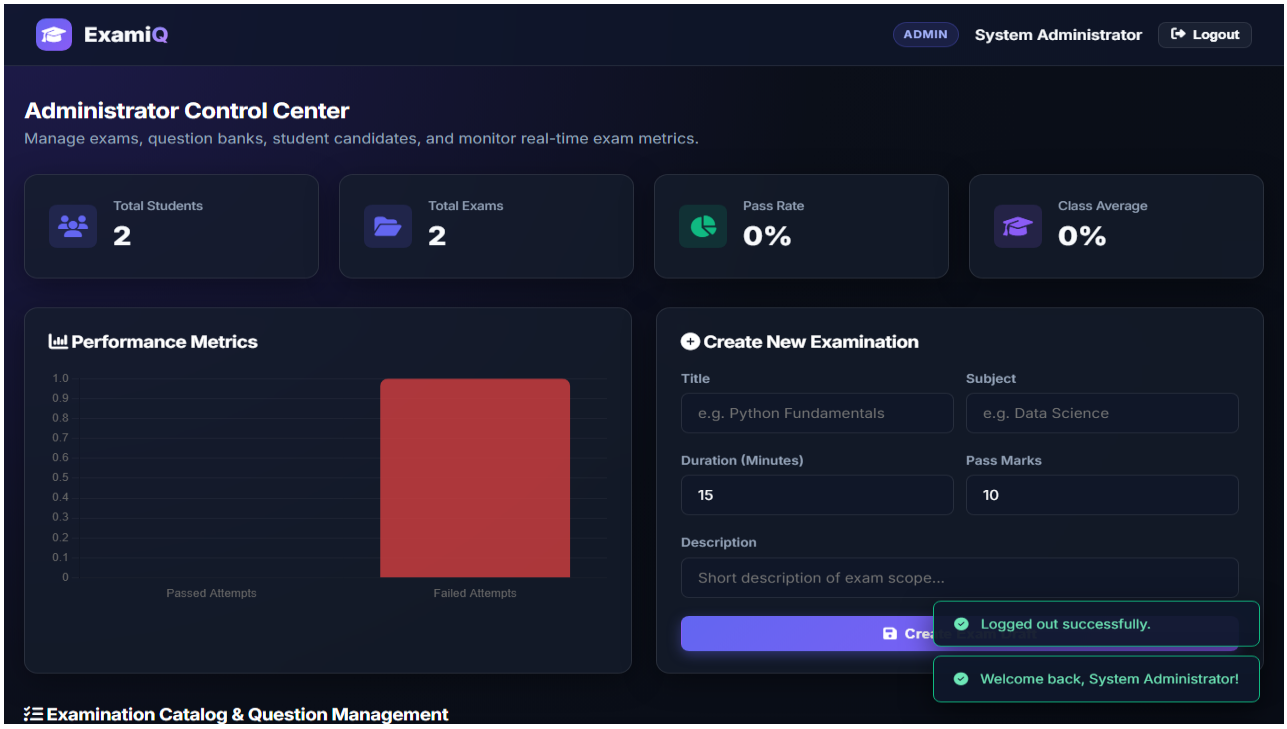
Proctored focus detection modal triggered when student switches browser tabs or loses window focus.

Figure 3.5: Automated Scorecard & Detailed Answer Review



Instant result evaluation view displaying percentage badge, score summary, explanation boxes, and PDF export button.

Figure 3.6: Administrator Control Center & Chart.js Metrics



Admin Dashboard featuring real-time class metrics, Chart.js performance visualization, exam creator, and question bank manager.

4. Core Source Code Snippets

4.1 Auto-Evaluation Engine (ExamService.java)

```
// Automated Evaluation Algorithm taking positive & negative marks into account
private AttemptResultDto evaluateAndFinalizeAttempt(ExamAttempt attempt, AttemptStatus targetStatus) {
    Exam exam = attempt.getExam();
    List<Question> questions = questionRepository.findByExamId(exam.getId());
    List<StudentAnswer> answers = studentAnswerRepository.findByAttemptId(attempt.getId());
    Map<Long, StudentAnswer> answerMap = answers.stream()
            .collect(Collectors.toMap(ans -> ans.getQuestion().getId(), ans -> ans));

    double scoreObtained = 0.0;
    for (Question q : questions) {
        StudentAnswer sa = answerMap.get(q.getId());
        Set<Long> correctOptionIds = q.getOptions().stream()
                .filter(QuestionOption::isCorrect)
                .map(QuestionOption::getId).collect(Collectors.toSet());

        if (sa == null || sa.getSelectedOptionIds() == null || sa.getSelectedOptionIds().isEmpty()) {
            continue;
        }

        if (sa.getSelectedOptionIds().equals(correctOptionIds)) {
            scoreObtained += q.getMarks();
            sa.setMarksAwarded(q.getMarks());
            sa.setCorrect(true);
        } else {
            double penalty = exam.isEnableNegativeMarking() ? exam.getNegativeMarksPerQuestion() : 0.0;
            scoreObtained -= penalty;
            sa.setMarksAwarded(-penalty);
            sa.setCorrect(false);
        }
        studentAnswerRepository.save(sa);
    }
    attempt.setScoreObtained(Math.max(0.0, scoreObtained));
    attempt.setSubmitTime(LocalDateTime.now());
    attempt.setPassed(scoreObtained >= exam.getPassMarks());
    attempt.setStatus(targetStatus);
    return getAttemptResult(attemptRepository.save(attempt).getId());
}
```

4.2 Anti-Cheat Security Violation Tracker (ExamService.java)

```
@Transactional
public void logViolation(Long attemptId) {
    ExamAttempt attempt = attemptRepository.findById(attemptId)
            .orElseThrow(() -> new IllegalArgumentException("Attempt not found"));
    attempt.setViolationCount(attempt.getViolationCount() + 1);

    // Auto submit if violation threshold (3 violations) reached
    if (attempt.getViolationCount() >= 3 && attempt.getStatus() == AttemptStatus.IN_PROGRESS) {
        attempt.setStatus(AttemptStatus.FLAGGED_VIOLATION);
        evaluateAndFinalizeAttempt(attempt, AttemptStatus.FLAGGED_VIOLATION);
    } else {
        attemptRepository.save(attempt);
    }
}
```

4.3 Server-Side OpenPDF Scorecard Generator (PdfReportService.java)

```
public byte[] generateAttemptReportPdf(AttemptResultDto result) {
    Document document = new Document(PageSize.A4, 36, 36, 36, 36);
    ByteArrayOutputStream out = new ByteArrayOutputStream();
    try {
        PdfWriter writer = PdfWriter.getInstance(document, out);
        document.open();
        Font headerFont = FontFactory.getFont(FontFactory.HELVETICA_BOLD, 22, new Color(79, 70, 229));
        document.add(new Paragraph("ExamiQ - Official Scorecard", headerFont));

        PdfPTable scoreTable = new PdfPTable(4);
        scoreTable.setWidthPercentage(100);
        addScoreBox(scoreTable, "Score Obtained", String.format("%.1f / %.1f", result.getScoreObtained(), res
ult.getTotalMarks()));
        addScoreBox(scoreTable, "Percentage", String.format("%.2f%%", result.getPercentage()));
        addScoreBox(scoreTable, "Result Status", result.isPassed() ? "PASSED" : "FAILED");
        document.add(scoreTable);
        document.close();
    } catch (Exception e) { e.printStackTrace(); }
    return out.toByteArray();
}
```

5. Installation & Execution Guide

Building the Executable JAR:

```
.\maven\apache-maven-3.9.9\bin\mvn.cmd clean package -DskipTests
```

Running the Server:

```
java -jar target\online-examination-system-1.0.0.jar
```

Accessing the Web Portal:

Navigate to **http://localhost:8080**

- Student Demo: student1 / pass123
- Admin Demo: admin / admin123

End of Project Documentation Report - ExamiQ Online Examination Platform